

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Лабораторная работа № 1 на тему
«Реализация REST API на основе boilerplate»**

Выполнил:

Суворин Иван

БР 2.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задание

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и [мой](#))
2. С учетом имеющихся данных по результатам Д31 и Д32 реализуйте:
 - модели
 - представления
 - контроллеры
3. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Ход работы

1. Исходные данные

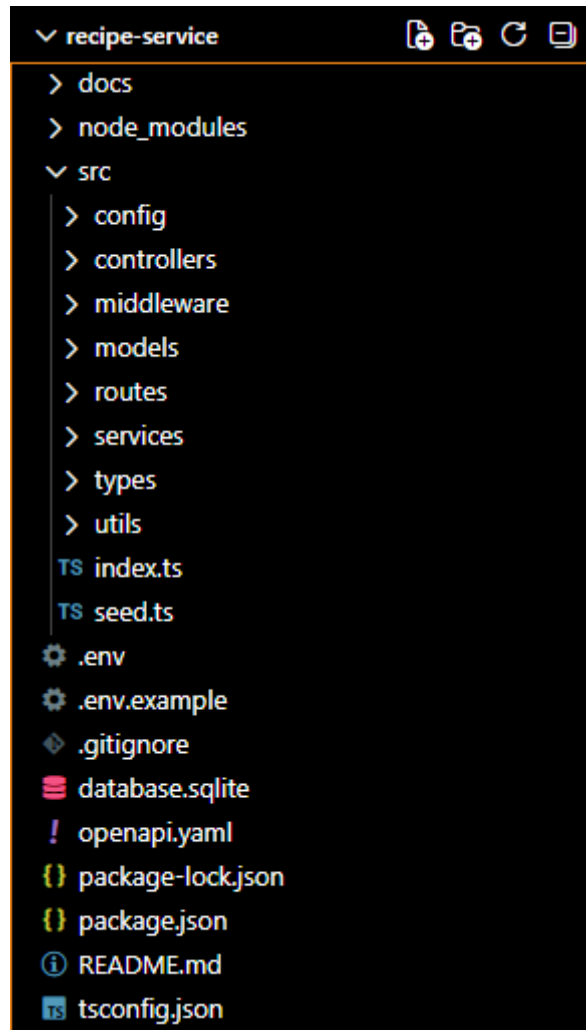
Было принято решение реализовывать версию из main.ts (согласно предыдущим правкам), так как она полнее покрывает функциональные требования, сформулированные ранее.

2. Технологический стек

- Node.js + TypeScript – среда выполнения и язык;
- Express – веб-фреймворк, маршрутизация и middleware;
- TypeORM – ORM для описания сущностей и работы с базой данных;
- SQLite (через драйвер sql.js) – база данных в виде одного файла, не требующая установки отдельного сервера СУБД;
- jsonwebtoken – авторизация по JWT (Bearer-токен), как и было спроектировано в Д32;
- bcryptjs – хэширование паролей;
- swagger-ui-express – интерактивная документация API (нефункциональное требование «Swagger» из Д32).

Структура проекта построена по мотивам boilerplate, предложенного на гитхабе (express-typeorm-boilerplate), но адаптирована под единое монолитное приложение.

3. Структура проекта



- config - конфигурация приложения и подключение к БД
- controllers - разбор запроса -> вызов сервиса -> ответ
- middleware - авторизация, обработка ошибок
- models - сущности TypeORM
- routes - маршруты Express
- services - бизнес-логика
- types – типы
- utils - JWT, пароли, валидация, сериализация ответов

Каждый запрос последовательно проходит цепочку route -> middleware (проверка авторизации) -> controller -> service -> TypeORM repository -> БД. Контроллеры не содержат бизнес-логики – она полностью сосредоточена в сервисах.

4. Реализованные модели данных

По результатам ДЗ1 реализовано 10 сущностей TypeORM. По сравнению с ERD-диаграммой было сделано одно осознанное упрощение: Ingredient и Step хранятся как записи, принадлежащие конкретному рецепту (а не как отдельный переиспользуемый справочник), поскольку именно так они описаны в контракте API (main.tsp) – как встроенные объекты рецепта.

Таблица 1 - Сущности

Сущность	Назначение	Ключевые поля и связи
User	Пользователь	username, email, passwordHash, role (user/admin), isEmailVerified; 1:N Recipe, 1:N Comment
Category	Категория рецепта	title; 1:N Recipe
Tag	Тег для фильтрации	title; N:N Recipe (через recipe_tags)
Unit	Справочник единиц измерения	name
Recipe	Рецепт	title, description, difficulty, cookingTime, status (draft/pending/published/rejected); N:1 User, N:1 Category, N:N Tag, 1:N Ingredient, 1:N Step, 1:N Comment, 1:N Like, 1:N Favorite
Ingredient	Ингредиент рецепта	name, quantity, unit; N:1 Recipe (принадлежит конкретному рецепту)
Step	Шаг приготовления	order (в БД – step_number), description; N:1 Recipe
Comment	Комментарий к рецепту	text; N:1 Recipe, N:1 User
Like	Лайк рецепта	составной первичный ключ (userId + recipeId) – гарантирует 1 лайк на пользователя
Favorite	Избранный рецепт	составной первичный ключ (userId + recipeId)

5. Реализованные эндпоинты API

Всего реализовано 26 операций, сгруппированных по ресурсам.

Таблица 2 - Auth

Запрос	Описание	Доступ
POST /auth/register	Регистрация нового пользователя	Публичный
POST /auth/login	Вход, выдает JWT-токен	Публичный
POST /auth/verify-email	Подтверждение email по токену	Публичный

Таблица 3 - Users

Запрос	Описание	Доступ
GET /users/me	Профиль текущего пользователя	Частный
PUT /users/me	Обновление профиля	Частный
GET /users/me/recipes	Свои рецепты (в любом статусе)	Частный
GET /users/me/favorites	Избранные рецепты	Частный

Таблица 4 - Recipes

Запрос	Описание	Доступ
GET /recipes	Список опубликованных рецептов: поиск, фильтры, сортировка, пагинация	Публичный
GET /recipes/{id}	Рецепт по id	Публичный
POST /recipes	Создание рецепта (статус draft)	Частный
PUT /recipes/{id}	Редактирование рецепта	Частный (автор)
DELETE /recipes/{id}	Удаление рецепта	Частный (автор/админ)
POST /recipes/{id}/submit	Отправка на модерацию	Частный (автор)
POST /recipes/{id}/like	Поставить лайк	Частный
DELETE /recipes/{id}/like	Убрать лайк	Частный
POST /recipes/{id}/favorite	Добавить в избранное	Частный
DELETE /recipes/{id}/favorite	Убрать из избранного	Частный

Таблица 5 - Admin

Запрос	Описание	Доступ
PUT /admin/recipes/{id}/approve	Одобрить рецепт (pending -> published)	Частный (admin)
PUT /admin/recipes/{id}/reject	Отклонить рецепт (pending -> rejected)	Частный (admin)
DELETE /admin/recipes/{id}	Удалить любой рецепт	Частный (admin)
DELETE /admin/comments/{id}	Удалить любой комментарий	Частный

Запрос	Описание	Доступ
		(admin)

Таблица 6 - Comments

Запрос	Описание	Доступ
GET /comments/recipe/{recipeId}	Список комментариев к рецепту	Публичный
POST /comments/recipe/{recipeId}	Добавить комментарий	Частный
DELETE /comments/{id}	Удалить комментарий	Частный (автор/админ)

Таблица 7 - Ingredients / Units

Запрос	Описание	Доступ
GET /ingredients?search=	Поиск по ингредиентам всех рецептов	Публичный
GET /units	Справочник единиц измерения	Публичный

6. Ключевые технические решения

— Жизненный цикл рецепта: draft -> (автор отправляет /submit) -> pending -> (решение администратора) -> published / rejected. Из rejected можно повторно отправить на модерацию.

— GET /recipes возвращает только опубликованные рецепты; одиночный рецепт по id виден всем, если он опубликован, иначе – только автору или администратору.

— Редактирование (PUT) доступно только автору; удаление – автору или администратору.

— Фильтр по cookingTime трактуется как «не более N минут».

— Лайк и избранное реализованы как честные отдельные POST/DELETE (а не переключатель); повторный лайк – 409, снятие несуществующего – 404. Уникальность лайка (1 пользователь – 1 лайк) гарантирована составным первичным ключом на уровне БД.

— Подтверждение email реализовано без реальной отправки писем: токен подтверждения выводится в консоль сервера при регистрации.

— Категории, теги и единицы измерения – справочники без отдельных CRUD-эндпоинтов; заполняются один раз скриптом npm run seed.

7. Документация API

Для выполнения нефункционального требования Swagger, сформулированного в Д32 подключен swagger-ui-express: интерактивная документация доступна по адресу /api-docs.

Демонстрация работы API

Базовый сценарий, подтверждающий работоспособность ключевого процесса приложения: регистрация/вход -> создание рецепта -> отправка на модерацию -> одобрение администратором -> появление в публичном списке.

1. Корректная обработка несуществующего маршрута

Обращение к корню приложения, под который не заведен отдельный обработчик, возвращает единый формат ошибки и корректный HTTP-статус 404 – как того требует нефункциональное требование «Обработка ошибок» из Д32.



Рисунок 1 – Ответ сервера на несуществующий маршрут

2. Публичный справочный эндпоинт

GET /api/units возвращает данные, ранее загруженные в БД скриптом npm run seed.

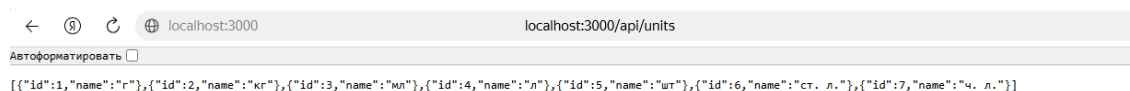


Рисунок 2 – Список единиц измерения из БД

3. Вход администратора

POST /auth/login с учетной записью администратора (создана скриптом npm run seed) возвращает JWT-токен.

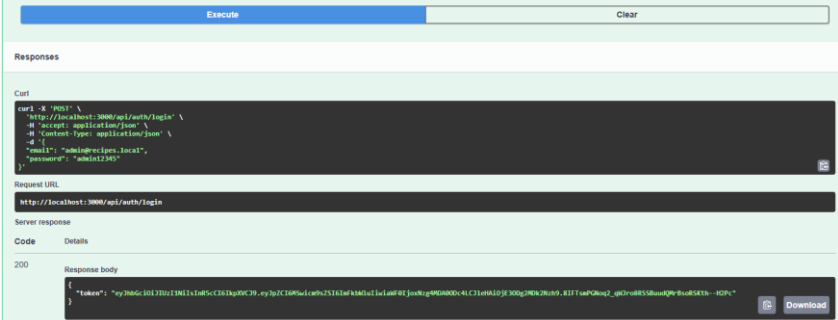
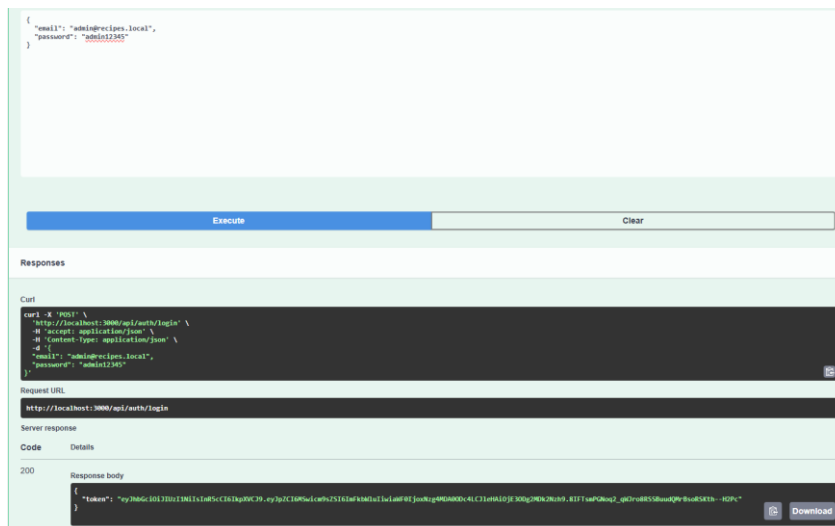


Рисунок 3 – Успешный вход, получение токена

4. Авторизация в Swagger UI

Полученный токен передан через кнопку Authorize – теперь запросы к защищенным эндпоинтам выполняются от имени администратора.

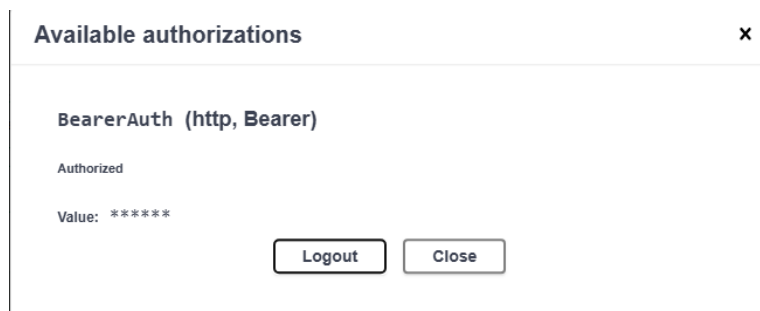


Рисунок 4 – Токен принят Swagger UI

Auth			^
POST	/auth/register	Регистрация нового пользователя	⌵
POST	/auth/login	Вход в систему	⌵
POST	/auth/verify-email	Подтверждение email по токenu (токен выводится в консоль сервера при регистрации)	⌵
Users			^
GET	/users/me	Получить профиль текущего пользователя	🔒 ⌵
PUT	/users/me	Обновить профиль текущего пользователя	🔒 ⌵
GET	/users/me/recipes	Рецепты, созданные текущим пользователем (в любом статусе)	🔒 ⌵
GET	/users/me/favorites	Избранные рецепты текущего пользователя	🔒 ⌵
Recipes			^
GET	/recipes	Список опубликованных рецептов с фильтрацией, сортировкой и пагинацией	⌵
POST	/recipes	Создать новый рецепт (статус draft)	🔒 ⌵
GET	/recipes/{id}	Получить рецепт по ID	⌵
PUT	/recipes/{id}	Обновить рецепт (только автор)	🔒 ⌵
DELETE	/recipes/{id}	Удалить рецепт (автор или админ)	🔒 ⌵
POST	/recipes/{id}/submit	Отправить рецепт на модерацию (draft/rejected -> pending)	🔒 ⌵
POST	/recipes/{id}/like	Поставить лайк	🔒 ⌵
DELETE	/recipes/{id}/like	Убрать лайк	🔒 ⌵
POST	/recipes/{id}/favorite	Добавить рецепт в избранное	🔒 ⌵
DELETE	/recipes/{id}/favorite	Удалить рецепт из избранного	🔒 ⌵
Comments			^
GET	/comments/recipe/{recipeId}	Список комментариев к рецепту	⌵
POST	/comments/recipe/{recipeId}	Добавить комментарий к рецепту	🔒 ⌵
DELETE	/comments/{id}	Удалить комментарий (автор или админ)	🔒 ⌵
Admin			^
PUT	/admin/recipes/{id}/approve	Одобрить рецепт (pending -> published)	🔒 ⌵
PUT	/admin/recipes/{id}/reject	Отклонить рецепт (pending -> rejected)	🔒 ⌵
DELETE	/admin/recipes/{id}	Удалить любой рецепт (только админ)	🔒 ⌵
DELETE	/admin/comments/{id}	Удалить любой комментарий (только админ)	🔒 ⌵
Ingredients			^
GET	/ingredients	Поиск ингредиентов (по всем рецептам)	⌵
Units			^
GET	/units	Список единиц измерения	⌵

Рисунок 5 - список всех групп эндпоинтов

5. Создание рецепта

POST /recipes с авторизацией создает рецепт со статусом draft, автором подставляется текущий пользователь:

Листинг 1:

```
{
  "id": 2,
  "title": "string",
  "shortDescription": "string",
  "fullDescription": "string",
  "difficulty": "easy",
  "cookingTime": 10,
  "author": {
    "id": 1,
    "username": "admin",
    "email": "admin@recipes.local",
    "firstName": "Админ",
    "lastName": "АДМИНОВ",
    "role": "admin"
  },
  "category": {
    "id": 1,
    "title": "Завтраки"
  },
  "tags": [
    {
      "id": 1,
      "title": "Быстро"
    }
  ],
  "ingredients": [
    {
      "name": "string",
      "quantity": "string",
      "unit": "string"
    }
  ],
  "steps": [
    {
      "order": 0,
      "description": "string"
    }
  ],
  "image": "string",
  "status": "draft",
  "likesCount": 0
}
```

Листинг 2 - ответ POST /recipes/1/submit

```
{
  "id": 2,
  "title": "string",
  "shortDescription": "string",
  "fullDescription": "string",
  "difficulty": "easy",
  "cookingTime": 10,
  "author": {
    "id": 1,
    "username": "admin",
```

```

    "email": "admin@recipes.local",
    "firstName": "Админ",
    "lastName": "Админов",
    "role": "admin"
  },
  "category": {
    "id": 1,
    "title": "Завтраки"
  },
  "tags": [
    {
      "id": 1,
      "title": "Быстро"
    }
  ],
  "ingredients": [
    {
      "name": "string",
      "quantity": "string",
      "unit": "string"
    }
  ],
  "steps": [
    {
      "order": 0,
      "description": "string"
    }
  ],
  "image": "string",
  "status": "pending",
  "likesCount": 0
}

```

200

Response body

```

{
  "id": 2,
  "title": "string",
  "shortDescription": "string",
  "fullDescription": "string",
  "difficulty": "easy",
  "cookingTime": 10,
  "author": {
    "id": 1,
    "username": "admin",
    "email": "admin@recipes.local",
    "firstName": "Админ",
    "lastName": "Админов",
    "role": "admin"
  },
  "category": {
    "id": 1,
    "title": "Завтраки"
  },
  "tags": [
    {
      "id": 1,
      "title": "Быстро"
    }
  ],
  "ingredients": [
    {
      "name": "string",
      "quantity": "string"
    }
  ]
}

```

Response headers

```

access-control-allow-origin: *
connection: keep-alive
content-length: 506
content-type: application/json; charset=utf-8
date: Sun, 30 Aug 2026 12:18:56 GMT
etag: W/"1fa-bn8kj3z8kqjfv9q93AQ25ACUj7to"
keep-alive: timeout=5
x-powered-by: Express

```

Responses

Code	Description	Links
200	Рецепт со статусом pending	No links

Рисунок 6 - статус рецепта сменился на pending

Листинг 2 – ответ PUT /admin/recipes/1/approve

```
{
  "id": 2,
  "title": "string",
  "shortDescription": "string",
  "fullDescription": "string",
  "difficulty": "easy",
  "cookingTime": 10,
  "author": {
    "id": 1,
    "username": "admin",
    "email": "admin@recipes.local",
    "firstName": "Админ",
    "lastName": "Админов",
    "role": "admin"
  },
  "category": {
    "id": 1,
    "title": "Завтраки"
  },
  "tags": [
    {
      "id": 1,
      "title": "Быстро"
    }
  ],
  "ingredients": [
    {
      "name": "string",
      "quantity": "string",
      "unit": "string"
    }
  ],
  "steps": [
    {
      "order": 0,
      "description": "string"
    }
  ],
  "image": "string",
  "status": "published",
  "likesCount": 0
}
```

Server response

Code	Details	
200	Response body <pre>{ "id": 2, "category": { "id": 1, "title": "Завтраки" }, "tags": [{ "id": 1, "title": "Быстро" }], "ingredients": [{ "name": "string", "quantity": "string", "unit": "string" }], "steps": [{ "order": 0, "description": "string" }], "image": "string", "status": "published", "likesCount": 0 }</pre> Response headers <pre>access-control-allow-origin: * connection: keep-alive content-length: 588 content-type: application/json; charset=utf-8 date: Sun, 30 Aug 2020 12:22:41 GMT etag: W/"1fc-7/AT08v1P4u6I4dQKav3Buzjp2Q" keep-alive: timeout=5 x-powered-by: Express</pre>	
Responses		
Code	Description	Links
200	Опубликованный рецепт	No links

Рисунок 7 - статус рецепта сменился на published

Листинг 3 – ответ GET /recipes

```
{
  "items": [
    {
      "id": 2,
      "title": "string",
      "shortDescription": "string",
      "fullDescription": "string",
      "difficulty": "easy",
      "cookingTime": 10,
      "author": {
        "id": 1,
        "username": "admin",
        "email": "admin@recipes.local",
        "firstName": "Админ",
        "lastName": "Админов",
        "role": "admin"
      },
      "category": {
        "id": 1,
        "title": "Завтраки"
      },
      "tags": [
        {
          "id": 1,
          "title": "Быстро"
        }
      ],
      "ingredients": [
        {
          "name": "string",
          "quantity": "string",
          "unit": "string"
        }
      ],
      "steps": [
        {
          "order": 0,
          "description": "string"
        }
      ],
      "image": "string",
      "status": "published",
      "likesCount": 0
    },
    {
      "id": 1,
      "title": "string",
      "shortDescription": "string",
      "fullDescription": "string",
      "difficulty": "easy",
      "cookingTime": 10,
      "author": {
        "id": 1,
        "username": "admin",
        "email": "admin@recipes.local",
        "firstName": "Админ",
        "lastName": "Админов",
        "role": "admin"
      },
      "category": {
        "id": 1,
        "title": "Завтраки"
      },
      "tags": [
        {
          "id": 1,
```

```
        "title": "Быстро"
    },
    ],
    "ingredients": [
        {
            "name": "string",
            "quantity": "string",
            "unit": "string"
        }
    ],
    "steps": [
        {
            "order": 0,
            "description": "string"
        }
    ],
    "image": "string",
    "status": "published",
    "likesCount": 0
}
],
"total": 2,
"page": 1,
"limit": 10
}
```

Проверка

```
POST /api/auth/login 200 102.694 ms - 169
POST /api/recipes 200 37.071 ms - 504
POST /api/recipes/2/submit 200 24.140 ms - 506
PUT /api/admin/recipes/2/approve 200 14.761 ms - 508
GET /api/recipes/2 200 6.311 ms - 508
GET /api/recipes?page=1&limit=10 200 6.759 ms - 1059
```

Рисунок 8 – проверка запросов через терминал

ВЫВОД

В рамках лабораторной работы №1 на основе структуры, предложенной в гитхабе, реализован REST API сервиса обмена рецептами: 10 сущностей БД, 26 эндпоинтов, авторизация по JWT, разграничение прав (пользователь/автор/администратор), полный жизненный цикл модерации рецепта, а также интерактивная документация Swagger.